

ECE 501 Project Week 3 Report

Project Title: Vehicle Orientation Estimation Using Conventional Image Processing

Group Number: 14

Project number: 8

Team Members:

Het Patel	AU2440221	het.p6@ahduni.edu.in
Shlok Shah	AU2440178	shlok.m2@ahduni.edu.in
Malay Panchal	AU2440050	malay.p@ahduni.edu.in
Dev Prajapati	AU2440022	dev.p12@ahduni.edu.in

Github Repository Link: https://github.com/hetpatel5107/DIP_2026_Project8_Group14

- **Introduction**

In Week 3, we started the actual image processing part of the project. So, this week's objective was to find gradient and edge information from a vehicle image using the Sobel operator. The cropped vehicle images obtained in Week 2 are used as the input for this week's work. First the cropped image was turned into grayscale. Then we calculated the Sobel gradients, along the X direction and the Y direction. After that we combined these gradients to find the magnitude. Finally we used the magnitude to figure out the dominant gradient direction.

For this week's demonstration, the cropped image v1_Sedan_17.1deg.jpg was used.



Actual data of v1_Sedan_17.1deg.jpg obtained from dataset of DRASHTI-HaOBB dataset:

1707 774 2022 871 2062 739 1747 642 Sedan 0 17.1

The heading value 17.1° in the filename represents the ground-truth orientation obtained from the dataset annotation in Week 2.

The overall process followed in Week 3 is:

Vehicle Crop → Grayscale → Sobel X & Y → Gradient Magnitude → Dominant Gradient Direction

- **GrayScale Conversion**

Code:

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("v1_Sedan_17.1deg.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap="gray")

plt.axis("off")

plt.show()
```

Explanation:

The cropped vehicle image is first loaded using OpenCV. The `cv2.cvtColor()` function with `cv2.COLOR_BGR2GRAY` converts the colour image into a grayscale image. This produces a single intensity value for each pixel, which is then used for gradient calculation in the next stages.

Output:



- **Sobel X and Sobel Y**

Code:

Sobel X

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

plt.imshow(abs(sobel_x), cmap="gray")

plt.axis("off")

plt.show()
```

Sobel Y

```
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

plt.imshow(abs(sobel_y), cmap="gray")

plt.axis("off")

plt.show()
```

Explanation:

The Sobel operator is used to detect changes in pixel intensity and identify edges in the vehicle image. Sobel X focuses on changes across the horizontal axis. Sobel Y focuses on changes across the vertical axis. Mathematically, Sobel X can be represented as $G_x = \partial I / \partial x$, and Sobel Y as $G_y = \partial I / \partial y$, where I represents the grayscale image.

X – Direction Kernel

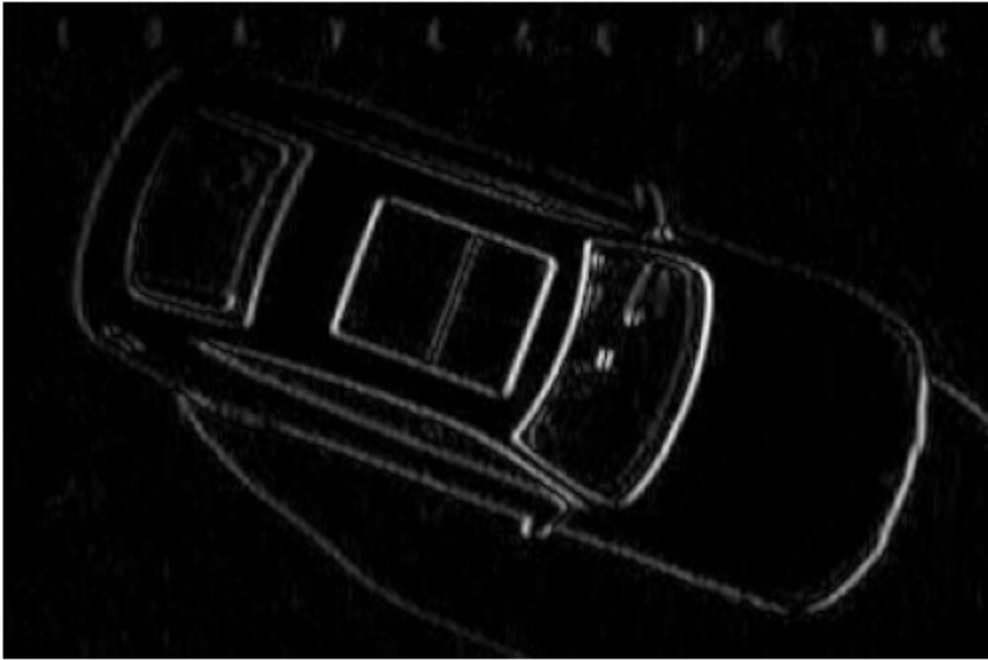
-1	0	1
-2	0	2
-1	0	1

Y – Direction Kernel

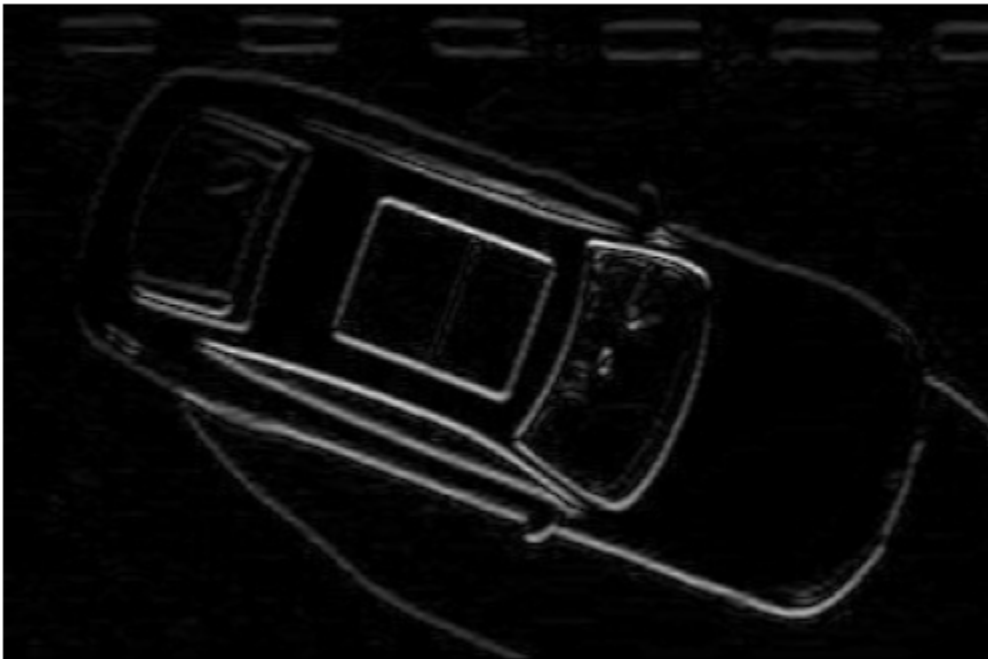
-1	-2	-1
0	0	0
1	2	1

In the above code, we use `cv2.Sobel()` to find where the vehicle edges are, separating them from the rest of the image. It does this by checking how the pixel brightness changes. The first operation uses the parameters 1, 0 to look for horizontal changes, which isolates the vehicle's vertical outlines. The second operation uses 0, 1 to look for vertical changes, which isolates the vehicle's horizontal outlines. We use `cv2.CV_64F` data precision to ensure that negative slope values aren't cropped during math operations, and then wrap the outputs in `abs()` inside the plotting function to convert those negative values into positive ones.

Output:
Sobel X



Sobel Y



- **Gradient Magnitude**

Code:

```
import numpy as np

gradient_magnitude = np.sqrt(sobel_x**2 + sobel_y**2)

gradient_magnitude = np.uint8(

    255 * gradient_magnitude / gradient_magnitude.max()

)

plt.imshow(gradient_magnitude, cmap="gray")

plt.axis("off")

plt.show()
```

Explanation:

After computing the gradient values for Sobel X and Y operations, the two gradient values are then added together to determine the gradient magnitude at each pixel. The gradient magnitude is the measure of the total intensity change in a particular pixel.

The gradient magnitude is calculated by:

$$G = \sqrt{(G_x^2 + G_y^2)}$$

where G_x is the Sobel X gradient and G_y is the Sobel Y gradient. The `np.sqrt()` function computes the square root of the squares of the two Sobel gradients. The reason for squaring the two gradients is that it gets rid of any sign associated with them.

The gradient magnitude obtained above is then normalized to the common 0 to 255 image intensity scale as follows:

```
gradient_magnitude = np.uint8(  
    255 * gradient_magnitude / gradient_magnitude.max()  
)
```

This normalization can be represented as:

$$G_{\text{normalized}} = 255 \times (G / G_{\text{max}})$$

where G_{max} is the maximum gradient magnitude of the image. This ensures that the maximum gradient value becomes 255, while the rest is mapped proportionally from 0 to 255. The `np.uint8()` function then converts the output to 8 bit images.

The last step involves displaying the gradient magnitude normalization using a grayscale colormap. The brighter the color, the higher the intensity difference (stronger edges), and vice versa.

Output:



In the above image output, the primary structural features of the car are captured by merging the information provided by Sobel X and Sobel Y. The areas of the boundaries of the car, roof,

window, wheel, and other regions that have high intensity variations will be more visible, whereas the rest of the region that is relatively smooth will be less visible.

- **Dominant Gradient Direction**

Code:

```
gradient_direction = np.degrees(np.arctan2(sobel_y, sobel_x))
angles = gradient_direction.flatten()
magnitudes = gradient_magnitude.flatten()
hist, bins = np.histogram(
    angles,
    bins=180,
    range=(-90, 90),
    weights=magnitudes
)
dominant_angle = bins[np.argmax(hist)]
print("Dominant gradient direction:", dominant_angle, "degrees")
```

Explanation:

After calculating the Sobel X and Y gradients, the gradient direction at each pixel is calculated using:

$$\theta = \tan^{-1}(G_y/G_x)$$

The `np.arctan2()` function calculates this angle for every pixel, while `np.degrees()` converts the values from radians to degrees. For example, a 100 × 100 vehicle image contains 10,000 pixels, so there can be 10,000 different gradient directions.

In order to find the single direction that dominates all other computed directions, all these directions are compiled in the form of a histogram using the `np.histogram()` function. Bins are created in the range of angle values between -90° and 90° , a total of 180 bins, with the gradient magnitudes as weights.

The bin that has the maximum gradient magnitude, obtained using the `np.argmax()` function, is selected, and the corresponding angle is taken as the dominating gradient direction.

For the selected sedan, the dominating gradient direction is found to be around 11° .

Output:

Dominant gradient direction: 11.0 degrees

- **Conclusion**

For this sedan, the orientation of the object is 17.1° , while the current Sobel gradient direction is around 11° .

Dataset: 1707 774 2022 871 2062 739 1747 642 Sedan 0 17.1

Actual orientation of the object: 17.1°

Output from Sobel operator: 11°

Hence, the absolute difference is:

$$|17.1 - 11| = 6.1 \text{ Degree}$$

The answer to this question is explained by the fact that the dominant direction among the gradients of the image does not necessarily coincide with the orientation of the car. This is because the gradients on the image are determined by the boundaries of the car, lighting conditions, the background, shadows, and other objects in the crop.

It is clear from the result obtained that the Sobel filter is able to obtain directional information about the car, but further processing is needed.

1. Work done this week

For this week, we moved on to the image processing phase by using the cropped images of vehicles from the DRASHTI-HaOBB dataset. The selected images of vehicles were first converted into grayscale images. The Sobel operator was then used to compute the gradient in both X and Y directions. After computing the magnitude of the gradient, the gradient direction and finally the dominant direction of the gradient was computed using weighted histograms. Initially an estimate of orientation was made and compared with the ground truth orientation given in the dataset.

2. Work to be done next week

Next week, the current Sobel-based method will be applied to multiple vehicle crops instead of a single vehicle. The estimated orientation for each vehicle will be compared with its ground-truth orientation, and the orientation error will be calculated. The results will be analysed to check the consistency of the method. Based on the observations, further conventional image-processing techniques will be explored to improve the orientation estimation.

3. Challenges faced

The key issue that needed to be resolved was the distinction between the gradient direction of the image pixels and the actual direction of the vehicle. The gradient direction may be influenced by the edges of the vehicle, shadows, brightness variations, and the background area. The preliminary results also indicate that there is a certain difference from the actual orientation of the vehicle, and therefore, additional tests are needed.